

Sri Lanka Institute of Information Technology

IT3060 – Human Computer Interaction

BSc (Hons) in IT 3rd Year 2nd Semester

Faculty of Computing – SLIIT

Lab Exercise 05 Report



FitFlow Redesign

Technology Selection & Architecture – Lab Exercise 05 Report

Student Name	MARASINGHE M M W K
IT No	IT23606760
Batch	Y3.S2.IT_WE_02.02
Case Study	FitFlow Redesign Scenario

Contents

Case Study Overview – FitFlow Redesign	3
Activity 1: Frontend Framework Comparison	4
Strengths and Weaknesses	4
Comparison Across Key Criteria	4
Suitability for FitFlow	5
Frontend Recommendation for FitFlow	5
Activity 2: Backend, Database and Authentication Comparison	6
Backend Frameworks	6
Database Options	6
Authentication and Authorization	7
Recommended Backend, Database and Auth Combination	7
Activity 3: Technology Comparison Matrix	8
Criteria and Weights	8
Weighted Frontend Decision Matrix	8
Recommended Technology Stack	9
Activity 4: High-Level Architecture Design	10
Key Components	10
Data Flows for Critical Features	11
Security, Scalability and Integration	11
Architecture Decision Records (ADR)	11
Activity 5: GitHub Repository Setup	12
Repository Folder Structure	12
README and Documentation	12
Repository Settings	12
First Commit and Repository Link	13

Case Study Overview – FitFlow Redesign

FitFlow is a fitness-tracking app from a mid-sized health-tech startup that had been steadily losing users, with app-store ratings falling from 4.6 to 3.8 stars and 68% of new users abandoning the app shortly after onboarding. Labs 01–04 carried the redesign through user research, data analysis and personas, design and prototyping, and moderated usability testing — where the selected “Variant B” prototype lifted the SUS score from 68 to 87. This report (Lab 05) moves from design into the technical foundation: it selects and justifies a full technology stack, designs a high-level system architecture, and sets up a version-controlled GitHub repository so the validated design can actually be built.

Every technology decision below is driven by FitFlow’s specific constraints — the three redesign priorities (AI-powered personalised workouts, social community features and improved nutrition tracking) and the non-functional requirements confirmed in Lab 02: fast performance, offline capability, accessibility, and strong data security under GDPR and CCPA.

Indicator	Before Redesign	Target / After Redesign
Average app-store rating	3.8 stars (down from 4.6)	Recover to ~4.5 stars
Users abandoning after onboarding	68%	Significantly reduced
User retention (past year)	Declining	Improve by ~35%
Usability score (SUS)	68 / 100 (current app)	87 / 100 (redesigned)

The chosen stack must therefore deliver a seamless iOS / Android / web experience, high performance, credible AI, real-time social features and compliant health-data handling — all quickly enough for a startup recovering lost ground.

Activity 1: Frontend Framework Comparison

Four leading options for the FitFlow mobile front end were analysed: Flutter, React Native, Kotlin Multiplatform (KMP) and Swift/SwiftUI. The case study requires a seamless iOS/Android/web experience with smooth animations, on-device AI and a fast path to market for a mid-sized startup — so the analysis weighs cross-platform reach and speed as heavily as raw performance.

Strengths and Weaknesses

Framework	Strengths	Weaknesses
Flutter	Single codebase across iOS, Android, web and desktop; own rendering engine gives excellent, consistent animations; hot reload; rich widget set.	Dart is a less common language (smaller talent pool); larger app size; AI/ML and some native features rely on community plugins.
React Native	JavaScript/React — largest ecosystem and talent pool; shares its language with the Node.js backend; mature Firebase, real-time and ML libraries; very fast development; backed by Meta.	Web needs React Native Web or a companion React app for a seamless result; heavy native features may need custom modules; older bridge overhead (much improved by the new architecture).
Kotlin Multiplatform	Shares business logic while keeping fully native UI → best-in-class native performance and platform feel; strong typing.	UI is still built per platform (more effort); younger ecosystem; web target still maturing; needs both iOS and Android skills.
Swift / SwiftUI	Best possible performance and integration on Apple devices; Core ML excels at on-device AI; clean modern UI.	iOS/Apple only — no Android and no web, so a whole second codebase is required, doubling cost and time.

Comparison Across Key Criteria

Criterion	Flutter	React Native	Kotlin MP	Swift/SwiftUI
Development speed	Strong	Excellent	Fair	Fair
Code reusability	Excellent (1 base)	Strong (~90%)	Good (logic only)	Weak (Apple only)
Performance	Excellent	Good → Strong	Excellent (native)	Excellent (native)
Ecosystem support	Good	Excellent	Fair	Strong (Apple)
Learning curve	Moderate (Dart)	Easy (JS/React)	Moderate	Moderate
Web compatibility	Strong	Fair (RN Web)	Limited (experimental)	None
AI / ML integration	Fair (plugins)	Strong (JS + native)	Fair	Excellent (iOS only)
Real-time features	Good	Excellent	Good	Good
Maintenance cost	Good	Low (shared JS)	Higher (2 UIs)	High (2 codebases)
Security	Good	Good	Strong	Strong

Suitability for FitFlow

Swift/SwiftUI is eliminated immediately: it cannot ship Android or web, which the case study explicitly demands. Kotlin Multiplatform offers the best native performance, but its per-platform UI and younger tooling slow a small team that needs speed. The real contest is Flutter versus React Native. Flutter has the edge on raw cross-platform rendering and web output, but React Native shares JavaScript/TypeScript with the planned Node.js backend (one language across the whole stack), has the largest talent pool (cheapest to hire and maintain for a mid-sized team), the most mature Firebase, real-time and ML integrations, and the fastest route to market — the startup's highest priority.

Frontend Recommendation for FitFlow

React Native is recommended for the FitFlow front end, with the company's web presence delivered through React Native Web or a lightweight companion React app that reuses the same components. This keeps a single JavaScript/TypeScript language across mobile, web and backend, and matches the front-end technology the case study already committed to. Flutter is recorded as a strong runner-up should raw rendering performance or a heavier web experience later become the deciding factor.

Activity 2: Backend, Database and Authentication Comparison

With the front end settled, the server side was evaluated across three layers — backend framework, database and authentication — each assessed for security and compliance (GDPR/CCPA and health data), real-time capability, AI integration, cost and maintainability for a mid-sized team.

Backend Frameworks

Framework	Assessment	Verdict
Node.js (Express / NestJS)	Event-driven — excellent for I/O-bound real-time features and notifications; shares JS/TS with React Native; huge ecosystem; NestJS adds structure, typing and testability.	Primary API + real-time layer
Python / FastAPI	Async and fast, with the strongest ML and data ecosystem (TensorFlow, PyTorch, scikit-learn) — ideal for the AI workout engine and computer-vision inference.	Dedicated AI microservice
Go	Compiled, outstanding concurrency and performance, but a smaller talent pool, weak ML ecosystem and more verbose code — overkill for a startup optimising for speed-to-market.	Not selected

Recommended backend: a NestJS (Node.js) core API handling auth, profiles, workout/nutrition CRUD, real-time social and notifications, paired with a Python/FastAPI AI microservice that isolates the machine-learning workload so it can scale and be re-deployed independently of the main app.

Database Options

Option	Scalability	Query performance	Health-data handling	Best fit
PostgreSQL	High (replicas)	Excellent (SQL, joins, analytics)	Excellent (ACID, row-level security, auditability)	Structured health data, logs, analytics
MongoDB	Very high (sharding)	Good (weaker joins)	Good (flexible, encrypted)	Flexible document data
Firebase Firestore	Very high (serverless)	Fair (limited joins)	Fair (managed, query/compliance limits)	Real-time feed, offline sync
DynamoDB	Very high	Fair (access-pattern-first)	Good (AWS compliance)	Massive key-value scale

Recommended database (hybrid): Firestore for the real-time social layer and offline sync — which directly satisfies the offline-capability requirement and the live community feed — plus PostgreSQL as the system of record for structured, query-heavy and compliance-sensitive health data (profiles, workout and nutrition history, analytics). This pairing answers all three lab criteria — scalability, query performance and health-data handling — better than any single database.

Authentication and Authorization

Option	Security & compliance	Ease	Cost	Stack fit
Firestore Auth	Strong (MFA, OAuth)	Excellent (drop-in)	Very low	Excellent (Firestore + RN)
AWS Cognito	Excellent (HIPAA-eligible)	Fair (complex)	Low	Good (AWS-centric)
Auth0	Excellent (certified)	Good	Higher at scale	Good (provider-agnostic)
Supabase Auth	Strong	Good	Low	Good (pairs with Postgres)

Recommended auth: Firestore Auth as the primary provider — email, social and phone sign-in with MFA, a first-class React Native SDK, near-zero cost and native fit with the Firestore real-time layer, giving the startup the fastest secure path to market. Sensitive health data is additionally protected behind the PostgreSQL layer’s own access controls, and Auth0 or Cognito can be layered in later if formal HIPAA certification becomes necessary.

Recommended Backend, Database and Auth Combination

Layer	Recommended Choice
Backend framework	NestJS (Node.js) core API + Python/FastAPI AI microservice
Database	Firestore (real-time / offline) + PostgreSQL (structured health data)
Authentication	Firestore Auth (MFA), with PostgreSQL access controls for health data

Activity 3: Technology Comparison Matrix

Findings from the frontend, backend, database and authentication comparisons were consolidated into a weighted decision matrix. The criteria weights reflect FitFlow's specific priorities established in Labs 01–02 — cross-platform reach, high performance for a fitness experience, deep AI integration, fast speed-to-market, real-time social features and strong health-data security.

Criteria and Weights

Criterion	Weight	Why it matters for FitFlow
Cross-platform reach	18%	Case study demands seamless iOS / Android / web
Performance	16%	Smooth fitness animations and responsiveness
AI / ML integration	14%	AI workouts and camera nutrition are core features
Development speed	12%	Startup needs a fast speed-to-market
Real-time features	10%	Live social feed, challenges and notifications
Security	10%	Health data under GDPR / CCPA
Ecosystem support	8%	Libraries, hiring and long-term support
Maintenance cost	7%	Mid-sized team with a limited budget
Learning curve	5%	Onboarding new developers quickly

Weighted Frontend Decision Matrix

Each framework was scored 1 (weak) to 5 (strong) on every criterion; the weighted total is the sum of score × weight, out of a maximum of 5.

Criterion	Weight	Flutter	React Native	Kotlin MP	Swift/UI
Cross-platform reach	0.18	5	4	3	1
Performance	0.16	5	4	5	5
AI / ML integration	0.14	3	5	3	5
Development speed	0.12	4	5	3	3
Real-time features	0.10	4	5	4	4
Security	0.10	4	4	5	5
Ecosystem support	0.08	4	5	3	4
Maintenance cost	0.07	4	5	3	2
Learning curve	0.05	3	5	3	3
Weighted Total (/5)	1.00	4.15	4.56	3.62	3.55

React Native scores highest at 4.56/5. It balances every FitFlow priority — strong cross-platform reach, good and improving performance, the most mature AI/ML and real-time ecosystems, the fastest development and the lowest maintenance cost through a shared JavaScript stack. Flutter is a close second (4.15) and the strongest alternative on raw performance and web rendering, while Kotlin MP (3.62) and Swift/SwiftUI (3.55) fall behind mainly on cross-platform reach and development speed for a small team.

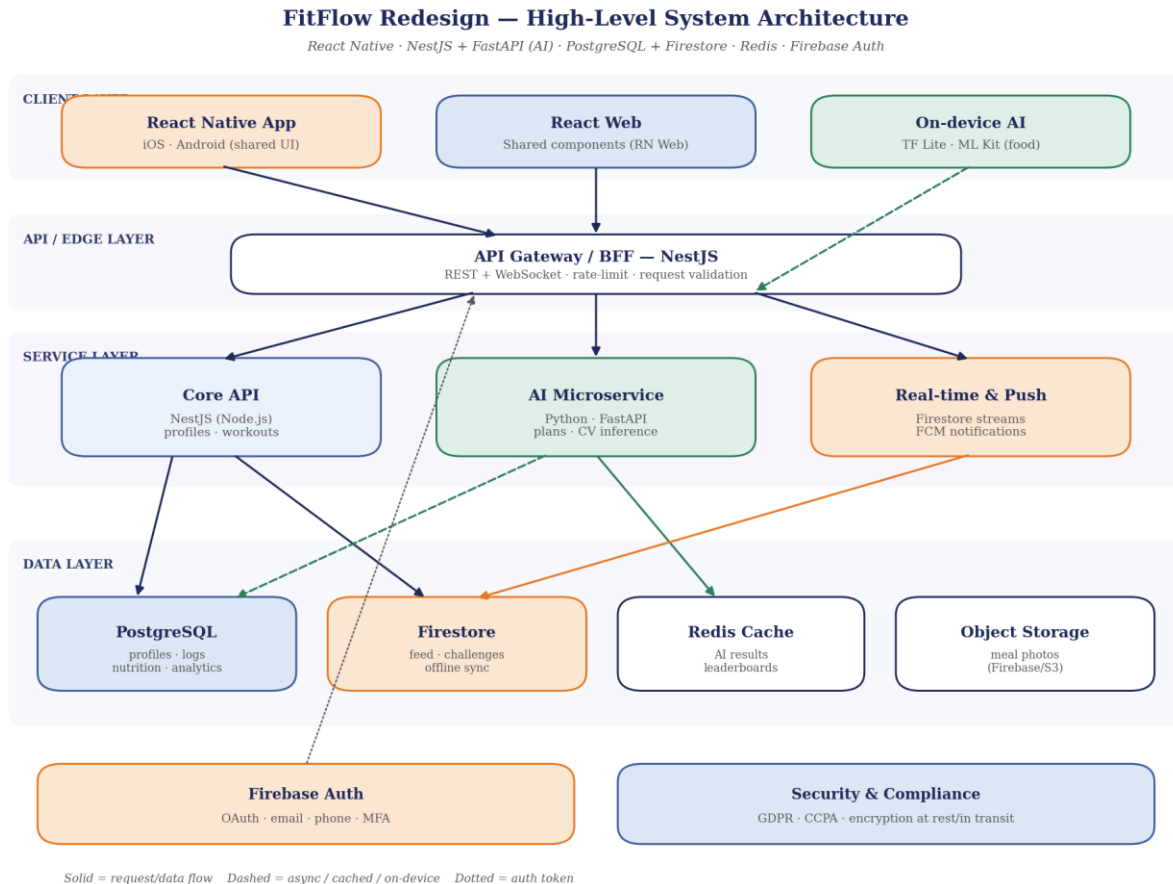
Recommended Technology Stack

Layer	Choice	Key reason
Frontend	React Native (+ RN Web)	One JS/TS language across the stack; largest ecosystem; fast to market
AI / ML	Python FastAPI + TensorFlow Lite / ML Kit	Best ML ecosystem for cloud models; on-device inference for speed & offline
Backend API	NestJS (Node.js)	Real-time-friendly, typed, shares language with the frontend
Real-time DB	Firestore	Live feed, challenges, offline sync and push
Relational DB	PostgreSQL	Structured, query-heavy, compliance-grade health data
Cache	Redis	AI-result caching, leaderboards, sessions
Authentication	Firebase Auth	Drop-in, low-cost, MFA, React Native-native
Storage	Firebase Storage / S3	Meal photos and media
Analytics	Firebase Analytics + Mixpanel	Carried over from the case study

This recommended stack matches the technologies the case study committed to — React Native, Node.js/Express, Firebase, TensorFlow Lite and ML-kit computer vision — and extends them with a clear AI microservice, a relational store for health data and a cache. Nothing in the design outruns what the team planned to build, keeping the whole redesign feasible within scope.

Activity 4: High-Level Architecture Design

The selected stack was assembled into a layered, service-oriented architecture. The diagram below shows the client, API/edge, service and data layers, with authentication and security as cross-cutting concerns. Solid arrows are request/data flows, dashed arrows are asynchronous, cached or on-device paths, and the dotted line is the authentication token.



FitFlow Redesign — high-level system architecture (original diagram)

Key Components

- **Client layer:** React Native app (iOS/Android) and a React web app sharing components; on-device TensorFlow Lite and ML Kit handle quick AI adaptations and camera food-recognition, working offline.
- **API / edge layer:** a NestJS API gateway / BFF exposes REST and WebSocket endpoints and handles routing, rate-limiting and request validation.
- **Service layer:** a Core API (NestJS) for profiles, workouts and nutrition; an AI microservice (FastAPI) for plan generation and cloud vision inference; and a real-time & push service (Firestore streams + FCM).
- **Data layer:** PostgreSQL as the system of record for structured/health data, Firestore for the real-time feed and offline sync, Redis for caching AI results and leaderboards, and object storage for meal photos.
- **Cross-cutting:** Firebase Auth issues tokens verified at the gateway, while encryption and GDPR/CCPA controls apply across every layer.

Data Flows for Critical Features

- **Personalized workout plans:** the app requests a plan → gateway → the AI microservice reads the user's goals and history from PostgreSQL → generates an adaptive plan → caches it in Redis → returns it; on-device TensorFlow Lite makes quick offline adjustments to the available time.
- **Social sharing:** the app posts to the community → written to Firestore → other members' apps receive the update live through Firestore listeners → FCM sends a push notification, with no polling required.
- **Nutrition tracking:** the app captures a meal photo → on-device ML Kit recognises the food instantly (offline) → the user confirms → the entry is saved to PostgreSQL and the photo to object storage → nutrition history becomes queryable for the progress dashboard.

Security, Scalability and Integration

- **Security:** Firebase Auth with JWTs verified at the gateway; TLS in transit and encryption at rest; PostgreSQL row-level security for health data; and the GDPR/CCPA consent and data-handling carried over from Lab 01.
- **Scalability:** stateless services scale horizontally behind the gateway; Firestore and Redis auto-scale; and the AI microservice scales independently so machine-learning load never slows the core app.
- **Integration:** a single BFF gives the client one consistent contract; the decoupled AI service lets models be retrained and deployed without touching the app; and Firebase/Mixpanel analytics feed the post-deployment monitoring described in the case study.

Architecture Decision Records (ADR)

ADR-001 — Cross-platform framework and overall stack

- **Status:** Accepted.
- **Context:** FitFlow must ship a seamless iOS/Android/web fitness experience quickly, with AI features and health-data compliance, on a mid-sized-team budget.
- **Decision:** build the client in React Native (+ RN Web), with a NestJS API, a Python/FastAPI AI microservice, a Firestore + PostgreSQL hybrid data layer, Redis cache and Firebase Auth.
- **Consequences:** (+) one JS/TS language across the stack, fastest time-to-market, mature ecosystem and clean AI/real-time separation; (–) React Native's web output is less seamless than Flutter's and needs RN Web.
- **Alternatives:** Flutter (strong runner-up, rejected on shared-language and ecosystem grounds); KMP and SwiftUI (rejected on cross-platform reach and development speed).

ADR-002 — Hybrid database (Firestore + PostgreSQL)

- **Status:** Accepted.
- **Context:** the app needs both live, offline-capable social data and structured, compliance-sensitive health data.
- **Decision:** use Firestore for real-time/offline social data and PostgreSQL as the system of record for health and analytical data.
- **Consequences:** (+) each workload uses the right tool — real-time sync versus rich querying and compliance; (–) two data stores to operate, mitigated by clear ownership boundaries (social vs. health data).

Activity 5: GitHub Repository Setup

A version-controlled GitHub repository, `fitflow-redesign`, was created to hold the redesign's code and documentation, with a clear folder structure and all the supporting documents produced across Labs 01–05.

Repository Folder Structure

The repository separates the three runnable services (frontend, backend, AI service) from shared documentation and CI configuration:

```
fitflow-redesign/
├── frontend/           # React Native (+ RN Web) app
│   ├── src/
│   └── package.json
├── backend/           # NestJS core API
│   ├── src/
│   └── package.json
├── ai-service/        # Python FastAPI AI microservice
│   ├── app/
│   └── requirements.txt
├── docs/              # Project documentation
│   ├── tech-stack-summary.md
│   ├── comparison-matrix.md
│   ├── architecture.png
│   └── ADR/
│       ├── ADR-001-stack.md
│       └── ADR-002-database.md
├── .github/workflows/ # CI/CD
│   └── ci.yml
├── .gitignore
└── README.md
```

README and Documentation

The README summarises the project, the chosen technology stack, and how to run each service, and links to the docs folder. The tech-stack summary, the comparison matrix, the architecture diagram and the two ADRs are all committed under `docs/` so every decision stays traceable — mirroring the requirement-traceability approach used in Lab 02.

Repository Settings

- **.gitignore:** covers Node, Python and React Native artefacts — `node_modules/`, `.env`, `__pycache__`, `build/` and platform build outputs.
- **Branch protection (main):** requires a pull-request review and a passing CI check before any merge.
- **CI/CD (optional):** a GitHub Actions workflow (`ci.yml`) installs dependencies, lints and runs tests for the frontend, backend and AI service on every push and pull request.

First Commit and Repository Link

All supporting documents from Labs 01–05 — the user research, personas and requirements, the wireframes and prototype, the usability report and this technology report — were added to docs/ and committed as the initial commit. The repository was created and pushed with:

```
git init
git add .
git commit -m "Initial commit: FitFlow redesign docs, stack & architecture"
git branch -M main
git remote add origin https://github.com/kavinduMarasinghe/fitflow-redesign.git
git push -u origin main
```

Repository link: [IT3060-HCI-fitflow-redesign](https://github.com/kavinduMarasinghe/fitflow-redesign.git)